Interrupt-Driven I/O

1 Basic Idea

Definition

Interrupt-driven I/O is an I/O method in which the CPU starts an operation, blocks the requesting process, runs another process, and later resumes I/O work when the device raises an interrupt.

- Programmed I/O wastes CPU time by busy waiting.
- Interrupt-driven I/O avoids most of that waiting.
- The CPU gives the device one item of work and then runs another process.
- When the device is ready again, it interrupts the CPU.

Core Point

Interrupt-driven I/O lets the CPU overlap device waiting time with useful work from other processes.

2 Printer Motivation

Example

Suppose a printer prints 100 characters per second and does not buffer characters.

$$\text{Time per character} = \frac{1 \text{ second}}{100 \text{ characters}} = 0.01 \text{ second} = 10 \text{ msec}$$

After each character is written, the printer needs about 10 msec before accepting the next character. In programmed I/O, the CPU would sit in a polling loop for those 10 msec. That is enough time to perform a context switch and run another process.

3 System Call Path

Initial Print Request

When a process asks to print a string, the OS starts the first character and then blocks the requesting process until the whole string has printed.

1. The user process makes a print system call.
2. The OS copies the user buffer into a kernel buffer.

3. The OS waits until the printer can accept a character.
4. The OS writes the first character to the printer data register.
5. The requesting process is blocked.
6. The scheduler chooses another process to run.

Code During Print System Call

```
copy_from_user(buffer, p, count);
enable_interrupts();

while (*printer_status_reg != READY)
    ;                                /* wait for first character only */

*printer_data_register = p[0];
scheduler();                         /* run another process */
```

4 Interrupt Handler Path

Printer Interrupt

When the printer finishes one character and can accept another, it generates an interrupt.

1. The interrupt stops the currently running process.
2. The CPU saves that process's state.
3. The printer interrupt-service procedure runs.
4. If more characters remain, the handler sends the next character.
5. If no characters remain, the handler unblocks the original user process.
6. The handler acknowledges the interrupt and returns.

Printer Interrupt-Service Procedure

```
if (count == 0) {
    unblock_user();
} else {
    *printer_data_register = p[i];
    count = count - 1;
    i = i + 1;
}

acknowledge_interrupt();
return_from_interrupt();
```

5 What Changes Compared with Programmed I/O

Feature	Programmed I/O	Interrupt-driven I/O
Waiting style	CPU polls until the device is ready.	CPU runs another process while waiting.
Device notification	CPU discovers readiness by checking status.	Device announces readiness by interrupt.
CPU use	CPU can be wasted in a busy loop.	CPU time can be used for other work.
Complexity	Simple.	More complex because interrupts and handlers are needed.

6 Cost and DMA

Main Cost

If an interrupt occurs for every character, the system may spend too much time saving state, running handlers, acknowledging interrupts, and returning.

DMA Idea

Direct Memory Access (DMA) reduces CPU involvement by letting a DMA controller feed data to the device instead of interrupting the CPU for every character.

Feature	Interrupt-driven I/O	DMA
Interrupt frequency	Often one interrupt per character or small unit.	Usually one interrupt per buffer.
CPU involvement	CPU sends each next character from the interrupt handler.	DMA controller performs the repeated transfer.
Main benefit	Avoids busy waiting.	Reduces interrupt overhead and frees the CPU more.
Possible drawback	Too many interrupts can waste CPU time.	DMA may be unnecessary for small transfers.

Final Point

Interrupt-driven I/O improves over programmed I/O by avoiding busy waiting, but DMA improves further by reducing interrupts from many small events to one larger completion event.